

Soccer Results Analytics

Spark Streaming and Machine Learning Mini-Project

ESP – Academic Year 2025–2026

Fatimetou Cheikh Brahim Cheikh Abdallahi

<https://github.com/fatimetou80/mini-project-spark-football.git>

1. Problem Presentation

The objective of this mini-project is to design a full data pipeline for Mauritanian league football results, from data collection to predictive analytics.

The core problem is that raw match data is split between historical static files and continuously updated web data, while useful analytics require:

- reliable historical preparation;
- near real-time stream processing;
- reproducible storage layers;
- and a simple predictive model evaluated on recent matches.

This project addresses that problem by combining Spark batch processing, Kafka streaming simulation, Spark Structured Streaming, Parquet persistence, and Spark MLlib modeling.

2. Chosen Architecture

The implemented architecture has five layers:

1. **Batch preparation layer** in `notebooks/prepare_data.ipynb`.
2. **Current-season web scraping enrichment** in `notebooks/prepare_data.ipynb`.
3. **Kafka event simulation** in `notebooks/src/producer.py`.
4. **Structured Streaming analytics** in `notebooks/src/stream_job.py`.
5. **Machine Learning training/evaluation** in `notebooks/train_model.ipynb`.

The data flow is:

historical CSV → cleaned historical Parquet → current-season scraped CSV → Kafka topic → streaming Parquet → ML workflow

3. Historical Dataset and Scraped Source

- Historical dataset: `data/rim_championnat_results_2007-2025.csv`
- Main columns used: `season`, `date`, `home_team`, `away_team`, `home_goals`, `away_goals`
- Scraped source: <http://www.soccerpunter.com/results/26572/Mauritania-Super-D1-2025-2026>
- Scraping method: `requests` + `BeautifulSoup`, keeping completed matches only (score pattern X-Y)

4. Data Schema

- season (string)
- date (date)
- home_team (string)
- away_team (string)
- home_goals (int)
- away_goals (int)
- total_goals (int)
- goal_difference (int)
- result (home_win / draw / away_win)
- match_year (int)

5. Transformations Applied

5.1 Historical preparation

- null filtering on critical columns;
- goal columns cast to integer;
- robust date parsing across multiple formats;
- creation of derived columns:
 - total_goals
 - goal_difference
 - result
 - match_year
- Parquet output in `data/outputs/prepared/historical/`

5.2 Current-season enrichment

- scraping of date, team, score;
- filtering for season 2025–2026;
- normalization of team names;
- schema alignment;
- export to `current_season_clean.csv`

6. Scraping and Enrichment Process

1. HTTP request to source page.
2. HTML parsing for completed matches.
3. Field extraction and cleaning.
4. Team normalization.
5. Export for ML evaluation.

7. Implemented Streaming Aggregations

- Kafka topic: `mauritania_matches`
- JSON parsing with schema
- event-time reconstruction
- watermarking (2 minutes) + windowing (1 minute)
- Parquet output with checkpointing

Aggregation implemented:

- total home goals and match count by home team and time window

Outputs:

- data/outputs/streaming/football_stats
- checkpoint: checkpoints/football_stats

8. ML Target and Features

Task: **match result classification**

- Target: label from result
- Features:
 - home_team index
 - away_team index
 - match_year
 - avg home goals
 - avg away goals
 - form_gap

9. Obtained Results

9.1 Logistic Regression

- Accuracy: 0.404
- F1-score: 0.311

9.2 Random Forest

- Accuracy: 0.424
- F1-score: 0.402

10. Discussion

10.1 Partitioning

Partitioning by season or year would improve performance.

10.2 Checkpointing

Checkpointing ensures recovery and streaming reliability.

10.3 Format Choices

Parquet enables efficient storage and processing.

10.4 Data Quality Limitations

- inconsistent team naming
- scraping fragility
- new teams

10.5 Project Limitations

- limited features
- single data source
- basic streaming aggregation

10.6 Possible Improvements

- additional streaming metrics
- better feature engineering
- model tuning
- richer analytics

11. Bonus Work Implemented

- form_gap feature
- model comparison (LogisticRegression vs RandomForestClassifier)
- visualizations in `train_model.ipynb`